เฉลยและสื่อทบทวน: CPE333 Operating Systems

วิชา Operating Systems | ขอบเขต Lecture 1--6 | พื้นฐาน: กำลังทบทวนตามบทเรียน | เป้าหมาย: เข้าใจแนวคิดและใช้เตรียมสอบกลางภาค

เอกสารนี้สรุปแนวคิดที่เกี่ยวข้องกับข้อสอบจำลอง ไม่ใช่คำแปลหรือบทสรุปครบทุกสไลด์ของ Lecture 1--6. ใช้เอกสาร Lecture เป็นแหล่งหลัก และระบุชื่อไฟล์กับหน้า PDF ไว้ในแต่ละหัวข้อและตารางแหล่งอ้างอิง

A. ภาพรวมและความเชื่อมโยงของบท

คำอธิบายเสริม: ระบบปฏิบัติการ (Operating System, OS) ทำให้โปรแกรมใช้ CPU และหน่วยความจำได้อย่างสะดวก โดยยังคงควบคุมและแยกการทำงานของแต่ละโปรเซสไว้. Lecture 1 วางพื้นฐานเรื่องสิทธิ์การทำงานและการป้องกัน; Lecture 2 อธิบายว่าโปรแกรมที่กำลังทำงานกลายเป็นโปรเซสและ OS สลับบริบทอย่างไร; Lecture 3 ใช้นโยบาย scheduling แบ่งเวลา CPU; ส่วน Lecture 4--6 ไล่จากการจัดสรรหน่วยความจำ ไปสู่ paging, virtual memory, page fault และการเลือก page ที่จะนำออก. ดังนั้นโจทย์ Scheduling ในข้อเขียนข้อ 1 และ page replacement ในข้อ 3 เป็นการนำกลไกเหล่านี้มาคำนวณจริง.

แหล่งหลัก: lectures/ Lecture1_IntroductionToOS.pdf, Lecture2_ProcessAndProcessAPIs.pdf, Lecture3_Scheduling.pdf, Lecture4_MemoryConceptAndContiguousMemoryAllocation.pdf, Lecture5_NoncontiguousMemoryAllocation.pdf และ Lecture6_VirtualMemory.pdf.

B. สรุปเนื้อหาแบบเข้าใจ

Lecture 1 -- OS, privilege และ isolation. OS จัดการทรัพยากรและทำให้โปรแกรมใช้งานได้ผ่าน abstraction. การแยก user mode กับ kernel mode ช่วยไม่ให้โปรแกรมทั่วไปสั่งงานที่มีสิทธิ์สูงโดยตรง; เมื่อโปรแกรมขอบริการจาก OS จะเข้าสู่ kernel ผ่านกลไก trap. ตัวอย่างคือ system call ที่ขอให้ OS ทำงานแทน. Trap เป็นกลไกส่งการควบคุม ส่วน system call คือบริการ/คำขอ จึงเกี่ยวข้องกันแต่ไม่ใช่คำเดียวกัน. แหล่ง: Lecture1_IntroductionToOS.pdf, pp. 2, 11, 13--15.

Lecture 2 -- process และ context switch. Program คือชุดคำสั่งที่ยังไม่ทำงาน; process คือโปรแกรมที่กำลังทำงาน พร้อมข้อมูลและสถานะควบคุม. OS เก็บรายละเอียดของแต่ละ process ใน PCB และสลับ CPU จาก process หนึ่งไปอีก process ด้วยการบันทึก/กู้คืน context. สถานะหลักที่ใช้ในแบบจำลองคือ ready, running และ blocked: process ต้องกำลัง running ก่อนจึงจะ block จากการรอ I/O ได้. fork() สร้าง child และ wait() ให้ parent รอ child จบ. แหล่ง: Lecture2_ProcessAndProcessAPIs.pdf, pp. 4--12, 22--24, 27, 29, 33--40.

Lecture 3 -- scheduling. Scheduler เลือกงานที่พร้อมใช้ CPU. FIFO ทำตามลำดับเข้าคิว; SJF เลือกงานที่ใช้เวลาสั้นกว่าและอาจทำให้ turnaround เฉลี่ยดีเมื่อรู้เวลางาน แต่เสี่ยง starvation; RR แบ่ง CPU เป็น time slice จึงตอบสนองงานแรก ๆ ได้ไวขึ้น แต่การสลับถี่มี overhead. MLFQ ใช้หลายคิวและปรับ priority ตามพฤติกรรมของงาน โดยมีกฎ boost ช่วยไม่ให้งานรอนานเกินไป. อย่าสับสน turnaround time (เวลาตั้งแต่มาถึงจนเสร็จ) กับ response time (เวลาตั้งแต่มาถึงจนได้ CPU ครั้งแรก). แหล่ง: Lecture3_Scheduling.pdf, pp. 6--18, 24--35.

Lecture 4 -- contiguous allocation. Base/bound ใช้ตรวจว่า virtual address อยู่ในช่วงที่ process ได้รับอนุญาตหรือไม่; ตรวจ bound ก่อน แล้วจึงบวก base เพื่อหา physical address. Fixed-size partition อาจเสียพื้นที่ภายในช่องที่จัดสรร (internal fragmentation); flexible-size partition ลดปัญหานี้ได้ แต่เกิดช่องว่างภายนอกที่นำไปใช้ต่อไม่ได้ (external fragmentation) และอาจต้อง compaction. คำว่า register ใน C เป็น storage class ไม่ได้รับประกันว่า compiler จะทำให้ตัวแปรเร็วที่สุด. แหล่ง: Lecture4_MemoryConceptAndContiguousMemoryAllocation.pdf, pp. 10--11, 17--18, 27--31, 34--35.

Lecture 5 -- segmentation และ paging. Segmentation แบ่ง address space ตามส่วนที่มีความหมาย;

paging แบ่ง logical memory เป็น page ขนาดเท่ากัน และ physical memory เป็น frame ขนาดเท่ากัน. Page table ใช้แปลเลข page ไปเป็น frame; TLB เป็น cache ของผลแปล address. ดังนั้น TLB miss แปลว่าไม่พบ translation ใน cache เท่านั้น ยังอาจพบ page ใน RAM ผ่าน page table ได้; จึงไม่เท่ากับ page fault. แหล่ง: Lecture5_NoncontiguousMemoryAllocation.pdf, pp. 5--10, 12--16, 20--22.

Lecture 6 -- virtual memory. Demand paging นำ page เข้ามาเมื่อมีการอ้างถึงและ page ยังไม่อยู่ใน RAM. Page fault handler ตรวจเหตุการณ์และนำ page เข้ามาก่อนเริ่มคำสั่งเดิมใหม่; หากไม่มี frame ว่างจึงต้องเลือก victim. FIFO ดูลำดับที่เข้า, LRU ดูการใช้งานครั้งล่าสุด, ส่วน OPT มองการใช้งานในอนาคตจึงเป็นเกณฑ์เปรียบเทียบเชิงอุดมคติ. EAT ขึ้นกับสัดส่วน hit/fault และเวลาเข้าถึงตามแบบจำลองใน Lecture; หาก page fault เกิดถี่เพราะมี frame ไม่พอ ระบบอาจเกิด thrashing. แหล่ง: Lecture6_VirtualMemory.pdf, pp. 9--14, 18--24, 29--34.

C. ศัพท์สำคัญ

English	คำไทยที่ใช้	ความหมายในวิชานี้
Operating System (OS)	ระบบปฏิบัติการ	ซอฟต์แวร์ที่จัดการทรัพยากรและให้บริการแก่โปรแกรม
Program / Process	โปรแกรม / โปรเซส	ชุดคำสั่งที่ยังไม่ทำงาน / โปรแกรมที่กำลังทำงานพร้อมสถานะ
Process Control Block (PCB)	บล็อกควบคุมโปรเซส	โครงสร้างเก็บข้อมูลสถานะและการจัดการของโปรเซส
Context switch	การสลับบริบท	บันทึกสถานะงานเดิมและกู้สถานะงานที่จะได้ใช้ CPU
Trap / System call	แทรป / การเรียกใช้บริการ OS	กลไกโอนการควบคุมเข้า OS / คำขอบริการจากโปรแกรม
Scheduler	ตัวจัดตารางงาน	ส่วนที่เลือก process จากคิวพร้อมทำงาน
Turnaround time	เวลาตั้งแต่มาถึงจนเสร็จ	$C_i - A_i$; completion time ลบ arrival time
Response time	เวลารอจนได้ CPU ครั้งแรก	$S_i - A_i$; first start time ลบ arrival time
Multilevel Feedback Queue (MLFQ)	คิวป้อนกลับหลายระดับ	scheduler ที่ปรับระดับ priority ของงานตามพฤติกรรม
Internal / External fragmentation	เศษ เหลือ ภายใน / ภายนอก	พื้นที่เหลือในช่องที่จองแล้ว / ช่องว่างระหว่างพื้นที่จัดสรร
Base / Bound	ฐาน / ขอบเขต	ค่าเริ่มต้นของ physical memory / ขนาดช่วง virtual address ที่อนุญาต
Segmentation	การแบ่งส่วน	แบ่ง address space ตามส่วน เช่น code, stack หรือ heap
Paging: page / frame	เพจ / เฟรม	หน่วย logical memory / หน่วย physical memory ที่มีขนาดเท่ากัน
Page table / TLB	ตารางเพจ / บัฟเฟอร์แปลที่อยู่	ตารางแปล page เป็น frame / cache ของ translation
Page fault	เพจฟอลต์	เหตุการณ์ที่อ้างถึง page ซึ่งยังไม่อยู่ใน physical memory
Page replacement	การแทนที่เพจ	เลือก page ที่อยู่ใน frame เพื่อนำออกเมื่อจำเป็นต้องมี frame
Thrashing	การสลับเพจถี่เกินไป	ภาวะที่ระบบเสียเวลาส่วนใหญ่ไปกับการย้าย page เข้า-ออก แทนการทำงาน
Effective Access Time (EAT)	เวลาเข้าถึงเฉลี่ย	เวลาเฉลี่ยตามโอกาส hit/fault และต้นทุนที่กำหนด

D. คำอธิบายเสริม: โพยทบทวนเร็ว (1 หน้า)

สถานะและ CPU: Ready = พร้อมแต่ยังไม่ได้ CPU; Running = กำลังใช้ CPU; Blocked = รอเหตุการณ์ เช่น I/O. การ block เกิดจากงานที่กำลัง running ไม่ใช่กระโดดจาก ready โดยตรง. PCB เก็บ state และ context; context switch คือการบันทึก/กู้สถานะ.

Scheduling: $T_i = C_i - A_i$ (turnaround), $R_i = S_i - A_i$ (response). FIFO ตามคิว; SJF เลือก burst สั้น; RR แบ่ง time slice; MLFQ เลือกคิว priority สูงก่อน, ใช้ RR ในคิวเดียวกัน, งานใหม่เริ่มบนสุด, ใช้ allotment ครบแล้วลดระดับ และ boost เป็นระยะ.

Base/bound: address ถูกต้องเมื่อ $0 \le VA < bound$; ถ้าถูกต้อง $PA = base + VA$. ตรวจขอบเขตก่อนบวก base.

Paging/VM: page อยู่ใน logical memory, frame อยู่ใน physical memory; page และ frame ขนาดเท่ากัน. TLB เก็บ translation ไม่ใช่ page contents. TLB miss ให้ตรวจ page table; page fault เกิดเมื่อ page ที่ต้องใช้ไม่อยู่ใน RAM.

Replacement/EAT: FIFO = เข้าเก่าก่อนออก; LRU = ไม่ได้ใช้นานสุดออก; OPT = จะถูกใช้อีกช้าที่สุดในอนาคตออก. $EAT = (1 - p_f)m_a + p_f d_a$ ตาม Lecture 6; แปลงหน่วยให้ตรงกันก่อนแทนค่า.

ข้อควรระวัง: response time ไม่ใช่เวลารอรวม; page fault ไม่เท่ากับ TLB miss; internal fragmentation อยู่ในพื้นที่ที่จองแล้ว ส่วน external fragmentation เป็นช่องว่างระหว่างพื้นที่; OPT ใช้ข้อมูลอนาคตจึงทำจริงไม่ได้.

E. คำอธิบายเสริม: คำถามฝึกทบทวน 10 ข้อ

ลองตอบก่อนเปิดเฉลยท้ายเอกสาร. คำถามชุดนี้เขียนใหม่จากหัวข้อใน Lecture 1--6 เพื่อทบทวนแนวคิด ไม่ใช่ข้อสอบเก่าที่คัดลอกมา.

1. อธิบายความต่างระหว่าง program กับ process ให้ได้ใจความหนึ่งประโยค.
2. process ที่อยู่ในสถานะ ready เปลี่ยนเป็น blocked ได้โดยตรงหรือไม่? อธิบายเส้นทางสถานะที่เกี่ยวข้อง.
3. ยกตัวอย่างข้อมูล CPU สองอย่างที่ต้องบันทึกตอน context switch และบอกว่า PCB ใช้ทำอะไร.
4. มีงาน A, B, C มาถึงพร้อมกันตามลำดับและใช้ CPU 4, 2, 1 ms ตามลำดับ จงเรียงงานแบบ FIFO และ SJF (ไม่แทรกงาน) แล้วหา average turnaround time ของแต่ละแบบ.
5. process หนึ่งมาถึงเวลา 2 ms ได้ CPU ครั้งแรกเวลา 5 ms และเสร็จเวลา 9 ms จงหา response time และ turnaround time.
6. MLFQ ใช้ priority queue อย่างไร และมีกลไกใดช่วยลดโอกาส starvation?
7. มี 3 frames และอ้างถึงหน้า $1, 2, 3, 1, 4$. เมื่ออ้างถึงหน้า 4, FIFO กับ LRU เลือก victim ต่างกันอย่างไร เพราะเหตุใด?
8. base = 16 KiB, bound = 4 KiB และ virtual address = 3000 bytes. address นี้ใช้ได้หรือไม่ และ physical address เป็นเท่าไร?
9. อธิบายว่า TLB miss ต่างจาก page fault อย่างไร และเหตุใดอย่างแรกจึงไม่จำเป็นต้องทำให้เกิดอย่างหลัง.
10. ถ้า $m_a = 10$ ns, $d_a = 10$ ms และ $p_f = 0.001$, จงหา EAT ตามสูตรใน Lecture 6 แล้วรายงานเป็น ns และประมาณเป็น μs.

เฉลยข้อสอบจำลอง: ส่วนที่ 1

สำหรับตรวจหลังทำข้อสอบเท่านั้น ไม่ใช่เฉลยข้อสอบจริงของรายวิชา. คะแนนรวม 50: ปรนัย $20c/41$ เมื่อ c คือจำนวนข้อถูก; เติมคำ 10 คะแนน; อัตนัย $8+4+8=20$ คะแนน. ไม่ปัดคะแนนรายข้อ; ปัดคะแนนรวมเป็นทศนิยมสองตำแหน่งเมื่อสรุปผล และไม่มีการหักคะแนนตอบผิด.

เฉลยส่วนที่ 1: คำตอบเดียวที่ดีที่สุด

คงข้อความโจทย์และตัวเลือกไว้ในเล่มข้อสอบตามแหล่งเดิม. เฉลยด้านล่างบอกทั้งเหตุผลและจุดที่ต้นฉบับอาจกำกวมโดยไม่แก้ถ้อยคำต้นฉบับเงียบ ๆ.

1. **(a)** ข้อนี้เป็นตัวเลือกที่ทำเครื่องหมายไว้ในข้อสอบเก่า. ตามหลักการ trap คือกลไกส่งการควบคุมไปยัง OS handler; system call อาจเรียกใช้ trap จึงเกี่ยวข้องกันแต่ไม่ใช่สิ่งเดียวกันพอดี. [L1 p.2; E09 Q1]

2. **(c)** หากไม่มีการแยก user mode กับ kernel mode โปรเซสอาจข้ามการป้องกันหน่วยความจำและเข้าถึงข้อมูลของโปรเซสอื่นได้. [L1 pp.2,14; E09 Q2]

3. **(e)** แบบจำลองใน Lecture 2 มอง process ว่าประกอบด้วย Program, Data, Control state และ Process state ดังนั้นทั้งสี่ข้อเป็นองค์ประกอบ. [L2 p.4; E09 Q4]

4. **(b)** Isolation คือการแยกแอปพลิเคชันออกจากกัน เพื่อป้องกันไม่ให้แอปหนึ่งเข้าถึงหรือรบกวนอีกแอปโดยไม่ได้รับอนุญาต. [L1 p.11; E09 Q5]

5. **(b)** ข้อสอบเก่าทำเครื่องหมาย Timer interrupt แต่โจทย์พิมพ์ว่า ``non-preemptive''. สองส่วนนี้ขัดกัน เพราะ timer interrupt ใช้ยึด CPU คืนในระบบ preemptive; เฉลยนี้คงคำตอบตามเครื่องหมายต้นฉบับและชี้ข้อขัดแย้งไว้. [L2 p.29; E09 Q6]

6. **(d)** Process descriptor หรือ PCB เป็นข้อมูลที่ OS ดูแลใน kernel space; ส่วน code, data และ stack เป็นพื้นที่ของ process. [L2 p.11; E09 Q7]

7. **(c)** ตามผังหน่วยความจำแบบที่ใช้สอน stack เติบโตสวนทางกับ heap. [L4 p.11; E09 Q8]

8. **(d)** process ต้องอยู่สถานะ running ก่อนจึงจะร้องขอ I/O หรือทำสิ่งที่ทำให้ต้อง block ได้; ไม่มีการเปลี่ยนจาก Ready ไป Blocked โดยตรง. เฉลยพิมพ์ของต้นฉบับที่เลือก ``reads a file'' ไม่สอดคล้องกับสถานะในโจทย์. [L2 pp.6--8; E09 Q9]

9. **(d)** ในแบบจำลองนี้ ทั้งการสร้าง process, การเสร็จของ I/O และการถูก timer interrupt แย่ง CPU ทำให้ process ไปอยู่ใน ready state ได้. [L2 pp.6--8,29,34; E09 Q10]

10. **(d)** Zombie คือ process ที่จบแล้ว แต่ยังมีรายการสถานะคงอยู่จนกว่า parent จะมาเก็บ exit status. [L2 pp.33,37--40; E09 Q12]

11. **(a)** PCB เก็บข้อมูลควบคุมของ process เช่น state และ CPU context ที่บันทึกไว้. [L2 p.11; E09 Q13]

12. **(a)** เป็นตัวเลือกที่ทำเครื่องหมายไว้ในข้อสอบเก่า. ให้อ่านว่า parent ยังมีชีวิตอยู่ หลัง child จบแต่ก่อน parent จะเก็บ exit status; ข้อความตัวเลือกเดิมย่อและไม่ชัดนัก. [L2 pp.37--40; E09 Q31]

13. **(e)** ตามโจทย์ lab ที่อ้างถึง parent จะพิมพ์หลังจาก child เสร็จ. อย่างไรก็ตาม การเรียก wait() เพียงอย่างเดียวไม่ได้ยืนยันว่า child พิมพ์ข้อความใด; ส่วนนั้นขึ้นกับโปรแกรม lab ที่ใช้. [L2 pp.37--38; E09 Q14]

14. **(d)** เมื่อเรียก fork() สำเร็จสองครั้ง จะมี process ทั้งหมด 4 ตัวที่ทำงานต่อและพิมพ์ข้อความคนละหนึ่งครั้ง. [L2 pp.34--35; E03 Q7]

15. **(e)** Timer interrupt, การร้องขอ I/O, การเรียก wait และการ yield ล้วนทำให้ process ปล่อย CPU ได้. [L2 pp.27,29; E09 Q16]

16. **(a)** ข้อสอบเก่าทำเครื่องหมาย FIFO. ให้ตีความว่า scheduler เลือกงานถัดไปตามลำดับที่มาถึง; ถ้าหมายถึงลำดับการได้ CPU ทุก time slice ตัวเลือก RR ก็อาจทำให้โจทย์อ่านได้อีกแบบ จึงควรระวังความกำกวมของ stem. [L3 p.7; E09 Q15]

17. **(b)** ใน storage class ของ C คำว่า register ใช้ระบุตัวแปร local ที่ร้องขอให้เก็บใน register; แต่ภาษา C ไม่รับประกันว่าตัวแปรนั้นจะเร็วที่สุดจริง. [L4 p.34; E09 Q19]

18. **(c)** Fixed-size partition อาจเหลือพื้นที่ไม่ได้ใช้ภายใน partition ที่จัดให้ process ซึ่งเรียกว่า internal fragmentation. ภาพข้อสอบเก่าทำเครื่องหมายข้อนี้; เฉลยตัวพิมพ์อีกชุดที่ตอบ none ไม่ตรงกับภาพ. [L4 pp.27,31; E09 Q20]

19. **(b)** Short-term scheduler เลือก process จาก ready queue เพื่อให้ได้ใช้ CPU. [L3 pp.3--4; E09 Q21]

20. **(c)** Page replacement เลือก page ที่อยู่ใน frame แล้วเพื่อนำออก เมื่อจำเป็นต้องใช้ frame นั้น. [L6 pp.13--24; E09 Q22]

21. **(e)** Paging ใช้ page table, segmentation ใช้ segment table และอาจใช้ TLB เร่งการแปล address; แต่ไม่ได้จำเป็นต้องใช้กลไกเหล่านี้พร้อมกันทุกแบบ. [L5 pp.8--9,14--15,22; E09 Q23]

22. **(d)** ข้อสอบเก่าทำเครื่องหมายข้อนี้และคงข้อความเดิมไว้. อย่างไรก็ดี (c) เรื่อง external fragmentation ก็เป็นข้อเสียของการจัดสรรแบบ contiguous ได้ จึงไม่ใช่คำถามที่มีคำตอบเดียวชัดเจนในเชิงเทคนิค. [L4 pp.29--31; L5 p.11; E09 Q24]

23. **(a)** Demand paging นำ page เข้ามาเมื่อมีการอ้างถึงและ page นั้นยังไม่อยู่ในหน่วยความจำหลัก. [L6 pp.9--12; E09 Q26]

24. **(e)** การบันทึก context, การเลือก victim เมื่อไม่มี frame ว่าง, การอ่าน page จาก disk และการปรับ page table ล้วนเป็นขั้นตอนที่อาจเกิดระหว่างจัดการ page fault ที่ถูกต้อง. [L6 pp.10--11; E09 Q27]

25. **(e)** ข้อสอบต้นฉบับทำเครื่องหมาย ``All choices"; ฟิลด์ที่ระบุคือ Base, Bound, Direction และ Valid. [L5 pp.8--9; E09 Q29]

26. **(c)** เมื่อขนาด address space คงที่ page ที่ใหญ่ขึ้นทำให้จำนวน page ลดลง จึงลดจำนวน entry ที่ต้องใช้ใน page table. [L5 pp.12--16; E09 Q30]

27. **(c)** RR แบ่งเวลา CPU ให้ process ที่พร้อมทำงานทีละช่วงสั้น ๆ จึงช่วยให้แต่ละงานได้เริ่มตอบสนองเร็วและแบ่งเวลาได้เป็นธรรมในระยะสั้น. [L3 pp.14--16; E09 Q33]

28. **(a)** FIFO ไม่ต้องแย่ง CPU ด้วย time slice; งานปัจจุบันทำต่อจนเสร็จหรือ block. [L3 p.7; E09 Q34]

29. **(e)** ข้อสอบเก่าทำเครื่องหมาย secondary storage ว่าเป็นที่เก็บภาพหน่วยความจำ; คงตัวเลือก ``virtual storage" ตามต้นฉบับ ไม่แก้ข้อความตัวเลือก. [L6 pp.5--8; E09 Q35]

30. **(c)** Response time คือเวลาตั้งแต่งานมาถึงจนได้ใช้ CPU ครั้งแรก. [L3 p.13; E09 Q36]

31. **(b)** Turnaround time เท่ากับ completion time ลบ arrival time. [L3 p.6; E09 Q37]

32. **(d)** ข้อสอบเก่าทำเครื่องหมาย ``Need compacting''. การ compact ช่วยรวบช่องว่างภายนอกที่เกิดกับ flexible-size partition ให้เป็นพื้นที่ต่อเนื่อง. [L4 pp.29--31; E09 Q38]

33. **(a)** ถ้าขนาด page และ page-table entry คงเดิม address space ที่ใหญ่ขึ้นต้องใช้ page-table entries มากขึ้น. [L5 p.16; E09 Q39]

34. **(e)** Multilevel page table ประหยัดพื้นที่ได้โดยไม่ต้องสร้างตารางระดับล่างสำหรับช่วง address ที่ไม่ได้ใช้. [L5 pp.17,19; E09 Q40]

35. **(a)** ใน state model แบบย่อที่ใช้ในบทเรียน process ใหม่จะเข้าสู่ ready state. [L2 p.6; E09 Q41]

36. **(b)** process ที่กำลังใช้ CPU อยู่ใน running state. [L2 p.6; E09 Q42]

37. **(e)** SJF อาจทำให้งานยาวรอนานมากหรือ starvation หากมีงานสั้นเข้ามาแซงอย่างต่อเนื่อง. [L3 pp.9--10; E09 Q44]

38. **(d)** RR ให้แต่ละงานที่พร้อมทำงานได้ใช้ CPU ใน time slice ของตน จึงช่วยลดเวลารอจนได้ตอบสนองครั้งแรก. [L3 pp.14--16; E09 Q46]

39. **(d)** Page เป็นหน่วยของ logical memory ส่วน page frame เป็นหน่วยของ physical memory และทั้งสองมีขนาดเท่ากัน. [L5 p.12; E09 Q48]

40. **(a)** OPT ใช้เป็นเกณฑ์อ้างอิงเชิงอุดมคติ เพราะต้องรู้ลำดับ page references ในอนาคตก่อนเลือก victim. [L6 p.22; E09 Q49]

41. **(a)** เป็นตัวเลือกที่ทำเครื่องหมายไว้ในข้อสอบเก่า. ให้อ่าน ``not valid in page table'' ว่า entry ของ virtual page ที่ถูกต้องระบุว่ายังไม่อยู่ใน RAM; หาก address นั้นผิดกฎหมายจริง จะเป็นคนละกรณีกับ page ที่ยังไม่ present และอาจเกิด protection fault. [L6 pp.9--11; E09 Q50]

เฉลยส่วนที่ 2: เติมคำ / ตอบสั้น

ข้อละ 1 คะแนน. ให้คะแนนตามองค์ประกอบที่ตอบถูก และยอมรับถ้อยคำอื่นที่มีความหมายเดียวกัน ไม่ต้องท่องประโยคเฉลยตามนี้ทุกคำ.

1. **MLFQ = Multilevel Feedback Queue; STCF = Shortest Time-to-Completion First.**
   ให้ 0.5 คะแนนต่อชื่อเต็มที่ถูกต้อง ยอมรับการเขียน Multi-level แบบมีขีดกลาง. [Lecture3_Scheduling.pdf, pp. 12, 24--35]

2. **Program คือชุดคำสั่งที่ยังไม่ทำงาน; process คือ program ที่กำลังทำงาน.**
   ให้ 0.5 คะแนนต่อใจความ. ไม่ต้องแจกแจงทรัพยากรหรืออธิบายเพิ่ม. [Lecture2_ProcessAndProcessAPIs.pdf, p. 4]

3. **MMU = Memory Management Unit; ใช้แปล virtual address เป็น physical address.**
   ให้ 0.5 คะแนนสำหรับชื่อเต็ม และ 0.5 คะแนนสำหรับหน้าที่แปล address. ไม่ต้องอธิบายเรื่อง protection เพิ่ม. [Lecture5_NoncontiguousMemoryAllocation.pdf, pp. 14--15]

4. **ตัวอย่าง: program counter (PC) และ stack pointer (SP).**
   ให้ 0.5 คะแนนต่อข้อมูล CPU ที่แตกต่างกัน เช่น PC, SP, general-purpose register หรือ processor status register. ไม่ต้องอธิบายลำดับการสลับงาน. [Lecture2_ProcessAndProcessAPIs.pdf, pp. 23--24]

5. **ตัวอย่าง: Process ID (PID) และ process state.**
   ให้ 0.5 คะแนนต่อข้อมูลคนละประเภทที่ไม่ใช่ค่า CPU register เช่น PID, state, scheduling information หรือ memory-management information. [Lecture2_ProcessAndProcessAPIs.pdf, p. 11]

เฉลยส่วนที่ 2: Short Answer (ต่อ)

6. **Zombie process.**
   รับคำว่า zombie หรือ zombie process ได้เต็ม 1 คะแนน; ไม่ต้องอธิบายเหตุผล และอย่าสับสนกับ orphan. [Lecture2_ProcessAndProcessAPIs.pdf, pp. 33, 37--40]

7. **I/O-intensive: คัดลอกไฟล์ขนาดใหญ่; CPU-intensive: คูณเมทริกซ์.**
   ให้ 0.5 คะแนนต่อประเภทงานที่ยกตัวอย่างและระบุถูกต้อง รับตัวอย่างอื่นที่สมเหตุสมผล ไม่ต้องนิยามหรือวิเคราะห์ burst. [Lecture3_Scheduling.pdf, pp. 19--21]

8. **Thrashing คือภาวะที่ระบบย้าย page เข้า-ออกระหว่าง RAM กับ disk ซ้ำ ๆ จนเสียเวลาส่วนใหญ่ไปกับการสลับ page.**
   ให้ 1 คะแนนเมื่ออธิบายการสลับ page ซ้ำได้ถูกต้อง; การบอกเพียงว่า page fault rate สูงยังไม่ใช่นิยามครบ. [Lecture6_VirtualMemory.pdf, pp. 33--34]

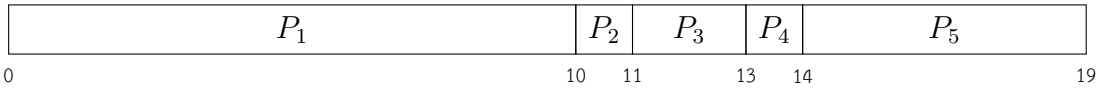
9. **ไม่ถูกต้อง เพราะ $4200 > 4096$ bytes ซึ่งเกิน bound.**
   ให้ 0.5 คะแนนสำหรับคำตอบ No และ 0.5 คะแนนสำหรับการเทียบกับ bound. ยังไม่ต้องบวก base หรือคำนวณ physical address. [Lecture4_MemoryConceptAndContiguousMemoryAllocation.pdf, pp. 17--18]

10. **TLB = Translation Lookaside Buffer; TLB miss อาจเกิดได้แม้ page ยังอยู่ใน RAM.**
    ให้ 0.5 คะแนนสำหรับชื่อเต็ม และ 0.5 คะแนนสำหรับแยก translation ที่ไม่อยู่ใน cache ออกจาก page ที่ไม่อยู่ใน RAM. [Lecture5_NoncontiguousMemoryAllocation.pdf, pp. 21--22; Lecture6_VirtualMemory.pdf, pp. 9--11]

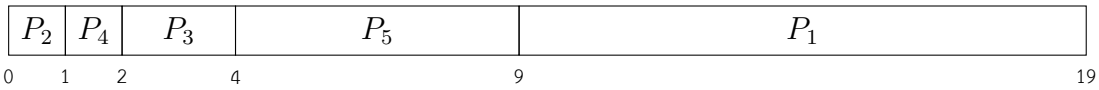
เฉลยอัตนัยข้อ 1: CPU Scheduling (8 คะแนน)

CPU burst ของ P_1--P_5 เท่ากับ 10, 1, 2, 1 และ 5 ms ตามลำดับ; ทุกงานมาถึงเวลา 0 และ RR ใช้ quantum 1 ms. การคำนวณนี้ไม่นับ overhead จากการสลับงาน. เมื่อ SJF เจองานที่ใช้เวลาเท่ากัน ให้เรียงตามลำดับมาถึงเพื่อทำเฉลย; นี่เป็นข้อตกลงในการคำนวณ ไม่ใช่เงื่อนไขที่เพิ่มลงในโจทย์. หากใช้ tie-break แบบอื่นที่ยังถูกต้องตาม SJF และคำนวณสอดคล้องกัน ให้รับคำตอบนั้นได้.

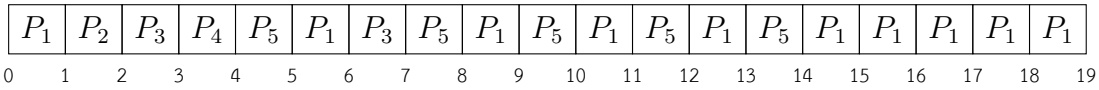
FIFO



SJF



RR



แม้ช่วงท้ายจะเหลือ P_1 งานเดียว RR ก็ยังแบ่งเวลาเป็น quantum ละ 1 ms. ใน Gantt chart จะวาดช่วง $[14, 19)$ เป็นกล่องเดียวได้ หากทำเครื่องหมายจุดแบ่ง quantum ที่ 15, 16, 17 และ 18 ms ไว้ด้วย.

นิยามที่ใช้

$T_i = C_i - A_i$ คือ turnaround time; $R_i = S_i - A_i$ คือ response time; $W_i = T_i - B_i$ คือเวลารอรวม เมื่อมี CPU burst เดียวและไม่มี I/O.

C คือเวลาจบงาน, A คือเวลามาถึง, S คือเวลาได้ CPU ครั้งแรก และ B คือ CPU burst.

Response time นับถึงครั้งแรกที่ได้ CPU เท่านั้น จึงไม่รวมช่วงที่ process กลับไปรอระหว่าง quantum ของ RR.

เฉลยอัตนัยข้อ 1: ไล่ FIFO และ SJF

FIFO: ทุก process มาถึงพร้อมกัน จึงใช้ลำดับตามโจทย์ P_1, P_2, P_3, P_4, P_5. FIFO ปล่อยให้งานปัจจุบันทำจนเสร็จ ดังนั้นเวลาจบของแต่ละงานหาได้จากผลรวม burst ก่อนหน้า:

$$0 + 10 = 10, \qquad 10 + 1 = 11, \qquad 11 + 2 = 13, \qquad 13 + 1 = 14, \qquad 14 + 5 = 19.$$

โปรเซส	Burst (ms)	ช่วงใช้ CPU	เริ่มครั้งแรก S_i	เวลาจบ C_i
P_1	10	$[0, 10)$	0	10
P_2	1	$[10, 11)$	10	11
P_3	2	$[11, 13)$	11	13
P_4	1	$[13, 14)$	13	14
P_5	5	$[14, 19)$	14	19

ตัวอย่างเช่น P_3 เริ่มเมื่อ P_1 และ P_2 จบแล้ว คือ $10 + 1 = 11$ ms; ใช้ CPU อีก 2 ms จึงจบที่ $11 + 2 = 13$ ms.

SJF แบบ ไม่ แทรก งาน: ทุก งาน พร้อม ตั้งแต่ $t = 0$ จึง เลือก burst ที่ สั้น ที่สุด ก่อน. ลำดับ คือ $P_2(1), P_4(1), P_3(2), P_5(5), P_1(10)$. P_2 และ P_4 ใช้เวลาเท่ากัน จึงใช้ลำดับมาถึงตัดสิน.

โปรเซส	Burst (ms)	ช่วงใช้ CPU	เริ่มครั้งแรก S_i	เวลาจบ C_i
P_2	1	$[0, 1)$	0	1
P_4	1	$[1, 2)$	1	2
P_3	2	$[2, 4)$	2	4
P_5	5	$[4, 9)$	4	9
P_1	10	$[9, 19)$	9	19

เช่น เมื่อ P_2 และ P_4 จบที่เวลา 2 ms งานสั้นที่สุดที่เหลือคือ P_3 จึงจบที่ $2 + 2 = 4$ ms. โจทย์นี้ใช้ FIFO และ SJF แบบไม่แทรกงาน; คอลัมน์ Priority ไม่เกี่ยวกับสองนโยบายนี้.

เฉลยอัตนัยข้อ 1 (ต่อ): ไล่ Round Robin

เริ่มจาก ready queue $[P_1, P_2, P_3, P_4, P_5]$. แต่ละงานได้ใช้ CPU ไม่เกิน 1 ms ต่อรอบ; ถ้ายังไม่เสร็จให้นำกลับไปต่อท้ายคิว. รอบแรก P_1 ใช้ไป 1 ms เหลือ 9 ms จึงกลับไปท้ายคิว ส่วน P_2 ใช้ 1 ms แล้วเสร็จ.

เวลา (ms)	Process	Burst ที่เหลือ	ผลหลังจบ quantum
$[0, 1)$	P_1	9	กลับไปท้ายคิว
$[1, 2)$	P_2	0	เสร็จที่ $t = 2$
$[2, 3)$	P_3	1	กลับไปท้ายคิว
$[3, 4)$	P_4	0	เสร็จที่ $t = 4$
$[4, 5)$	P_5	4	กลับไปท้ายคิว
$[5, 6)$	P_1	8	กลับไปท้ายคิว
$[6, 7)$	P_3	0	เสร็จที่ $t = 7$
$[7, 8)$	P_5	3	กลับไปท้ายคิว
$[8, 9)$	P_1	7	กลับไปท้ายคิว
$[9, 10)$	P_5	2	กลับไปท้ายคิว
$[10, 11)$	P_1	6	กลับไปท้ายคิว
$[11, 12)$	P_5	1	กลับไปท้ายคิว
$[12, 13)$	P_1	5	กลับไปท้ายคิว
$[13, 14)$	P_5	0	เสร็จที่ $t = 14$
$[14, 15)$	P_1	4	เหลืองานเดียวในคิว
$[15, 16)$	P_1	3	ทำต่ออีกหนึ่ง quantum
$[16, 17)$	P_1	2	ทำต่ออีกหนึ่ง quantum
$[17, 18)$	P_1	1	ทำต่ออีกหนึ่ง quantum
$[18, 19)$	P_1	0	เสร็จที่ $t = 19$

เมื่อเหลือ P_1 เพียงงานเดียว มันยังถูกจัดสรร CPU ครั้งละ 1 ms. ใน Gantt chart รวมช่วงท้าย $[14, 19)$ เป็นกล่องเดียวได้ แต่ควรแสดงจุดแบ่งที่ 15, 16, 17 และ 18 ms ไว้.

เฉลยอัตนัยข้อ 1 (ต่อ): คำนวณเวลาและค่าเฉลี่ย

ใช้ $T_i = C_i - A_i$ สำหรับ turnaround time และ $R_i = S_i - A_i$ สำหรับ response time โดย S_i คือเวลาได้ CPU ครั้งแรก. เนื่องจากทุก process มาถึงเวลา $A_i = 0$, turnaround จึงเท่ากับเวลาจบงาน ส่วน response เท่ากับเวลาเริ่มครั้งแรก.

นโยบาย	โปรเซส	S_i	C_i	$T_i = C_i$	$R_i = S_i$
FIFO	P_1	0	10	10	0
	P_2	10	11	11	10
	P_3	11	13	13	11
	P_4	13	14	14	13
	P_5	14	19	19	14
SJF	P_1	9	19	19	9
	P_2	0	1	1	0
	P_3	2	4	4	2
	P_4	1	2	2	1
	P_5	4	9	9	4
RR	P_1	0	19	19	0
	P_2	1	2	2	1
	P_3	2	7	7	2
	P_4	3	4	4	3
	P_5	4	14	14	4

ตัวอย่างแทนค่า

FIFO: P_5 จบที่ 19 ms จึงมี $T_5 = 19 - 0 = 19$ ms และเริ่มใช้ CPU ครั้งแรกที่ 14 ms จึงมี $R_5 = 14 - 0 = 14$ ms.

RR: P_3 จบที่ 7 ms แม้ใช้ CPU รวมเพียง 2 ms จึงมี $T_3 = 7 - 0 = 7$ ms; ส่วนเวลาเริ่มครั้งแรกคือ 2 ms ทำให้ $R_3 = 2 - 0 = 2$ ms. การกลับไปรอหลังได้ CPU ครั้งแรกไม่ถูกรวมใน response time.

เฉลยอัตนัยข้อ 1 (ต่อ): ค่าเฉลี่ยและเกณฑ์ตรวจ

คำนวณค่าเฉลี่ย

- FIFO: $\overline{T} = (10 + 11 + 13 + 14 + 19)/5 = 67/5 = 13.4$ ms; $\overline{R} = (0 + 10 + 11 + 13 + 14)/5 = 48/5 = 9.6$ ms.
- SJF: $\overline{T} = (19 + 1 + 4 + 2 + 9)/5 = 35/5 = 7.0$ ms; $\overline{R} = (9 + 0 + 2 + 1 + 4)/5 = 16/5 = 3.2$ ms.
- RR: $\overline{T} = (19 + 2 + 7 + 4 + 14)/5 = 46/5 = 9.2$ ms; $\overline{R} = (0 + 1 + 2 + 3 + 4)/5 = 10/5 = 2.0$ ms.

สรุปข้อ (d): สำหรับชุดงานนี้ SJF ให้ turnaround time เฉลี่ยต่ำสุดที่ 7.0 ms ส่วน RR ให้ response time เฉลี่ยต่ำสุดที่ 2.0 ms. ผลนี้ขึ้นกับ burst และเวลามาถึงชุดนี้ ไม่ได้หมายความว่านโยบายเดียวกันจะดีที่สุดกับทุก workload.

เกณฑ์ตรวจ 8 คะแนน

- (a) 3 คะแนน: FIFO, SJF และ RR อย่างละ 1 คะแนน พิจารณาลำดับงานและเวลาบน Gantt chart.
- (b) 2 คะแนน: turnaround time ทั้ง 15 ค่า ให้คะแนนตามการใช้ $C_i - A_i$ อย่างสม่ำเสมอ.
- (c) 2 คะแนน: response time ทั้ง 15 ค่า โดยใช้เวลาได้ CPU ครั้งแรก $S_i - A_i$ ไม่ใช่เวลารอรวม.
- (d) 1 คะแนน: เลือก SJF สำหรับ turnaround time เฉลี่ยต่ำสุด (0.5) และ RR สำหรับ response time เฉลี่ยต่ำสุด (0.5).

หากคำนวณพลาดจุดหนึ่ง แต่ใช้ค่าที่ได้ต่ออย่างถูกต้องในขั้นถัดไป ให้คะแนนวิธีทำต่อเนื่อง และไม่หักความผิดเดิมซ้ำ.

เฉลยอัตนัยข้อ 2: MLFQ กับธนาคาร (4 คะแนน)

คำอธิบายเสริม: ตัวอย่างคำตอบตามโจทย์

> ช่วงลูกค้าแน่น ธนาคารอาจใช้ MLFQ โดยให้ลูกค้าใหม่เริ่มที่คิวความสำคัญสูง และเลือกบริการจากคิวที่มี priority สูงสุดก่อน หากลูกค้าอยู่คิวเดียวกันให้ผลัดกันใช้ช่องบริการแบบ RR. รายการที่ยังไม่เสร็จหลังใช้เวลาในคิวนั้นครบโควตาจึงเลื่อนไปคิวล่าง ซึ่งอาจกำหนดช่วงบริการให้ยาวขึ้น พร้อมเลื่อนผู้ที่รอนานกลับขึ้นคิวบนเป็นระยะเพื่อลดโอกาสถูกทิ้งค้าง. การคัดแยกรายการและเตรียมเอกสารล่วงหน้าช่วยลดเวลาที่ช่องบริการหยุดรอข้อมูล; การเพิ่มช่องบริการควรอิงกับภาระงานและกำลังคนจริง. ธุรกรรมควรหยุดได้เฉพาะจุดปลอดภัยและต้องบันทึกความคืบหน้า. จากนั้นจึงใช้เวลาที่วัดได้มาปรับนโยบาย โดยดูทั้งเวลาเฉลี่ยและความเป็นธรรม.

เทียบกับกฎของ MLFQ

1. **เลือกคิว priority สูงสุดก่อน.** ถ้า Q0 ยังมีลูกค้ารอ ให้บริการ Q0 ก่อน Q1 หรือ Q2; คิวล่างได้ใช้ช่องบริการเมื่อคิวที่สูงกว่าว่าง.
2. **ใช้ RR ภายในคิวเดียวกัน.** ลูกค้าในระดับเดียวกันได้ใช้ช่องบริการเป็นรอบ ๆ จึงไม่ปล่อยให้รายเดียวครองบริการตลอด.
3. **ให้ลูกค้าใหม่เริ่มคิวบนสุด.** ระบบยังไม่รู้เวลาที่แต่ละธุรกรรมต้องใช้ล่วงหน้า; งานสั้นอาจเสร็จในคิวบน ส่วนงานยาวจะใช้โควตาในคิวสูงต่อเนื่องมากกว่า.
4. **ใช้โควตาเวลา (time allotment) เพื่อลดระดับ.** ถ้ารายการยังไม่เสร็จหลังใช้เวลาสะสมครบโควตาของระดับนั้น ให้ย้ายลงหนึ่งคิว. การพักหรือ yield ก่อนหมด time slice ไม่ควรล้างเวลาที่ใช้ไปแล้ว.
5. **ทำ priority boost เป็นระยะ.** เลื่อนรายการที่ยังไม่เสร็จกลับขึ้นคิวบนตามรอบ เพื่อลดโอกาสที่งานซับซ้อนจะถูกงานสั้นแซงจนแทบไม่ได้รับบริการ.

ตัวอย่างการทำงานของคิว (ตัวเลขต่อไปนี้สมมติขึ้นเพื่ออธิบาย ไม่ใช่ข้อมูลจากโจทย์)
สมมติมีช่องบริการเดียว และลูกค้า A, B, C มาถึงพร้อมกัน ใช้เวลาทำรายการ 1, 5 และ 3 นาทีตามลำดับ. Q0 ให้เวลา 2 นาทีต่อครั้ง ส่วน Q1 ให้ 4 นาที.

เวลา (นาที)	คิว/ลูกค้า	เวลาที่เหลือ	ผล
0--1	Q0 / A	0 นาที	A เสร็จที่นาที 1
1--3	Q0 / B	3 นาที	ใช้ Q0 ครบ 2 นาที จึงย้ายไป Q1
3--5	Q0 / C	1 นาที	ใช้ Q0 ครบ 2 นาที จึงย้ายไป Q1
5--8	Q1 / B	0 นาที	B ใช้ต่ออีก 3 นาทีและเสร็จที่นาที 8
8--9	Q1 / C	0 นาที	C ใช้ต่ออีก 1 นาทีและเสร็จที่นาที 9

เมื่อลูกค้าทั้งหมดมาถึงเวลา 0, turnaround time ของ A, B, C คือ 1, 8 และ 9 นาทีตามลำดับ. ตัวอย่างนี้แสดงว่าคิวสูงช่วยให้งานสั้นเสร็จเร็ว ส่วนการลดระดับยังเปิดโอกาสให้งานยาวทำต่อจนจบ.

คำอธิบายเสริม: ข้อเสนอปรับปรุงและข้อจำกัด

- ตรวจเอกสารและคัดประเภทบริการก่อนเข้าคิว เพื่อลดเวลาที่ช่องบริการต้องหยุดรอข้อมูล; เพิ่มผู้เชี่ยวชาญหรือช่องบริการเฉพาะช่วงที่ข้อมูลชี้ว่าคุ้มกับกำลังคน.
- กำหนด time slice ของคิวบนให้สั้นพอที่งานง่ายจะเริ่มได้เร็ว แต่ไม่สั้นจนเสียเวลาสลับงานมากเกินไป; คิวล่างอาจให้ช่วงยาวขึ้นเพื่อลด overhead.
- บันทึกเวลามาถึง เริ่มบริการ พัก กลับมาทำต่อ และเสร็จงาน แล้วใช้ turnaround time ที่วัดได้จริงมาปรับ time slice, allotment และรอบ boost.
- ธุรกรรมธนาคารไม่สามารถหยุดสลับได้ทุกจุดเหมือน CPU จึงควรกำหนดจุดพักที่ปลอดภัยและบันทึกความคืบหน้า. การเพิ่ม counter ช่วยทำงานขนาน แต่มีต้นทุนด้านบุคลากรและพื้นที่.

MLFQ ไม่ได้รับประกันว่าจะทำให้ turnaround time เฉลี่ยต่ำที่สุดสำหรับทุกชุดงาน เพราะระบบไม่รู้เวลาบริการล่วงหน้า. ควรประเมินทั้งเวลารอ ความเป็นธรรม และความถูกต้องของธุรกรรม.

เกณฑ์ตรวจ 4 คะแนน: อธิบายและเทียบกฎ MLFQ 2 คะแนน; เสนอนโยบายที่นำไปใช้ได้ 1 คะแนน; เสนอวิธีปรับปรุงพร้อมเหตุผลหรือข้อจำกัด 1 คะแนน. รับตัวอย่างอื่นที่สอดคล้อง ไม่บังคับจำนวนคิวหรือเวลาให้ตรงกับตัวอย่างนี้. [Lecture3_Scheduling.pdf, pp. 24--35; โจทย์อ้างอิง ยำรวมมิตรข้อเขียน.pdf, pp. 4--6, 10]

เฉลยอัตนัยข้อ 3: Page Replacement (8 คะแนน)

ตาราง FIFO, LRU และ OPT (6 คะแนน)

เริ่มจาก frame ว่าง 3 ช่อง. แต่ละคอลัมน์แสดงสถานะหลังอ้างถึง page แล้ว. H (hit) หมายถึง page อยู่ใน RAM อยู่ก่อน จึงไม่ต้องเปลี่ยน frame; F (fault) หมายถึง page ยังไม่อยู่ใน frame จึงต้องนำเข้ามา. เครื่องหมาย -- แสดงว่า frame ยังว่าง. ตารางเหตุผลด้านล่างไล่เฉพาะครั้งที่ต้องเลือก page ที่จะนำออก (victim). [Lecture6_VirtualMemory.pdf, pp. 13--24, 29; โจทย์อ้างอิง ยำรวมมิตรข้อเขียน.pdf, p. 10]

FIFO: 13 faults, 7 hits

Ref. no.	1	2	3	4	5	6	7	8	9	10
Page	7	0	1	3	0	3	2	4	2	7
Frame 1	7	7	7	3	3	3	3	3	3	7
Frame 2	--	0	0	0	0	0	2	2	2	2
Frame 3	--	--	1	1	1	1	1	4	4	4
H/F	F	F	F	F	H	H	F	F	H	F

Ref. no.	11	12	13	14	15	16	17	18	19	20
Page	0	3	2	1	2	0	1	4	0	1
Frame 1	7	7	2	2	2	2	2	4	4	4
Frame 2	0	0	0	1	1	1	1	1	1	1
Frame 3	4	3	3	3	3	0	0	0	0	0
H/F	F	F	F	F	H	F	H	F	H	H

FIFO: เหตุผลที่เลือก page ออก Queue เรียงจาก page ที่เข้าก่อนสุดไปหลังสุด. การ hit ไม่ทำให้ลำดับ FIFO เปลี่ยน. การอ้างอิงครั้งที่ 1--3 ใช้เติม frame ให้ครบก่อนเริ่มแทนที่.

Reference no.	Page เข้า	Victim	Queue หลังแทนที่ (เก่า $\rightarrow$ ใหม่)
4	3	7	$[0, 1, 3]$
7	2	0	$[1, 3, 2]$
8	4	1	$[3, 2, 4]$
10	7	3	$[2, 4, 7]$
11	0	2	$[4, 7, 0]$
12	3	4	$[7, 0, 3]$
13	2	7	$[0, 3, 2]$
14	1	0	$[3, 2, 1]$
16	0	3	$[2, 1, 0]$
18	4	2	$[1, 0, 4]$

มี page fault 3 ครั้งระหว่างเติม frame ครั้งแรก และอีก 10 ครั้งที่ต้องแทนที่ page รวมเป็น 13 faults; อีก 7 references เป็น hits.

เฉลยอัตนัยข้อ 3: LRU และเหตุผลเลือก victim

LRU (Least Recently Used) เลือกแทนที่ page ที่ไม่ได้ถูกใช้นานที่สุด ณ ตอนเกิด page fault. ทุกครั้งที่ hit ต้องอัปเดตว่า page นั้นเพิ่งถูกใช้ จึงต่างจาก FIFO ซึ่งลำดับไม่เปลี่ยนเมื่อ hit.

LRU: 13 faults, 7 hits

Ref. no.	1	2	3	4	5	6	7	8	9	10
Page	7	0	1	3	0	3	2	4	2	7
Frame 1	7	7	7	3	3	3	3	3	3	7
Frame 2	--	0	0	0	0	0	0	4	4	4
Frame 3	--	--	1	1	1	1	2	2	2	2
H/F	F	F	F	F	H	H	F	F	H	F

Ref. no.	11	12	13	14	15	16	17	18	19	20
Page	0	3	2	1	2	0	1	4	0	1
Frame 1	7	7	2	2	2	2	2	4	4	4
Frame 2	0	0	0	1	1	1	1	1	1	1
Frame 3	2	3	3	3	3	0	0	0	0	0
H/F	F	F	F	F	H	F	H	F	H	H

ตรวจการเลือก page ออกของ LRU ตัวเลขในคอลัมน์สุดท้ายคือลำดับครั้งล่าสุดที่ page นั้นถูกอ้างถึงก่อนเกิด fault; เลือก page ที่ถูกใช้นานที่สุดแล้ว หรือมีเลขน้อยที่สุด.

Reference no.	Page เข้า	Victim	ครั้งล่าสุดของ victim
4	3	7	1
7	2	1	3
8	4	0	5
10	7	3	6
11	0	4	8
12	3	2	9
13	2	7	10
14	1	0	11
16	0	3	12
18	4	2	15

ตัวอย่างที่ reference 8: frames มี 3, 0, 2 และถูกใช้ครั้งล่าสุดที่ลำดับ 6, 5, 7 ตามลำดับ จึงนำ page 0 ออกเพราะไม่ได้ถูกใช้นานที่สุด. รวมกับ page fault 3 ครั้งแรกที่เติม frame จะได้ 13 faults และ 7 hits.

เฉลยอัตนัยข้อ 3: ตาราง OPT (ต่อ)

OPT นำ page ที่จะถูกใช้อีกครั้งช้าที่สุดในอนาคตออก; ถ้า page ใดไม่ถูกใช้อีกเลย ก็ถือว่าการใช้งานครั้งต่อไปอยู่ไกลที่สุด. วิธีนี้ต้องรู้ reference string ที่เหลือ จึงใช้เป็นเกณฑ์เปรียบเทียบมากกว่าจะนำไปใช้จริง.

OPT: 10 faults, 10 hits

Ref. no.	1	2	3	4	5	6	7	8	9	10
Page	7	0	1	3	0	3	2	4	2	7
Frame 1	7	7	7	7	7	7	7	7	7	7
Frame 2	--	0	0	0	0	0	0	4	4	4
Frame 3	--	--	1	3	3	3	2	2	2	2
H/F	F	F	F	F	H	H	F	F	H	H

Ref. no.	11	12	13	14	15	16	17	18	19	20
Page	0	3	2	1	2	0	1	4	0	1
Frame 1	0	0	0	0	0	0	0	0	0	0
Frame 2	4	3	3	1	1	1	1	1	1	1
Frame 3	2	2	2	2	2	2	2	4	4	4
H/F	F	F	H	F	H	H	H	F	H	H

ไล่การตัดสินใจเมื่อ OPT เกิด fault

ครั้งที่อ้างอิง 1--3 ใช้เติม frame ว่าง. เลขหลัง @ คือลำดับครั้งถัดไปที่ page นั้นจะถูกอ้างถึง; คำว่า ``ไม่มี" หมายถึงไม่พบ page นั้นอีกในส่วนที่เหลือ.

Ref. no.	Page เข้า	Frames ก่อน fault	การใช้ครั้งถัดไปของ resident pages	Victim
4	3	$7, 0, 1$	7@10, 0@5, 1@14	1
7	2	$7, 0, 3$	7@10, 0@11, 3@12	3
8	4	$7, 0, 2$	7@10, 0@11, 2@9	0
11	0	$7, 4, 2$	7 ไม่มี, 4@18, 2@13	7
12	3	$0, 4, 2$	0@16, 4@18, 2@13	4
14	1	$0, 3, 2$	0@16, 3 ไม่มี, 2@15	3
18	4	$0, 1, 2$	0@19, 1@20, 2 ไม่มี	2

ตัวอย่างที่ reference 4: ใน frames $7, 0, 1$, page 7 จะถูกใช้อีกที่ลำดับ 10, page 0 ที่ลำดับ 5 และ page 1 ที่ลำดับ 14. จึงนำ page 1 ออกเพราะต้องรอนานที่สุด. OPT เกิด fault 3 ครั้งตอนเติม frame และอีก 7 ครั้งตามตาราง รวม 10 faults; ที่เหลือ 10 ครั้งเป็น hits. ถ้ามี page หลายหน้าที่ไม่ถูกใช้อีกเลย จะเลือกหน้าใดออกก็ได้และยังให้ผล optimal.

เฉลยอัตนัยข้อ 3 (ต่อ): EAT และเกณฑ์ตรวจ

ขั้นที่ 1: แปลงหน่วยให้ตรงกัน

เวลาเข้าถึงหน่วยความจำ $m = 10$ ns $= 0.00001$ ms. เวลาเข้าถึง disk $D = 10$ ms $= 10{,}000{,}000$ ns. คำนวณโดยใช้ ns ทั้งหมดหรือ ms ทั้งหมดก็ได้ แต่อย่านำ ns ไปบวกกับ ms โดยตรง.

ขั้นที่ 2: หาโอกาส hit และ page fault

มีการอ้างถึง page ทั้งหมด 20 ครั้ง. ให้ $p =$ จำนวน faults$/20$ เป็นโอกาสเกิด page fault และ $h =$ จำนวน hits$/20 = 1 - p$ เป็นโอกาส hit. ใช้สูตรจาก Lecture 6:

$$\text{EAT} = hm + pD = (1-p)m + pD.$$

ในข้อนี้ D ใช้แทนเวลาเข้าถึง disk ตามที่โจทย์กำหนด และไม่บวกต้นทุนอื่นที่โจทย์ไม่ได้ให้มา.

ขั้นที่ 3: แทนค่าในสูตร

- **FIFO:** $p = 13/20 = 0.65$ และ $h = 7/20 = 0.35$.
  $\text{EAT} = 0.35(10\,\text{ns}) + 0.65(10{,}000{,}000\,\text{ns})$
  $= 3.5 + 6{,}500{,}000 = 6{,}500{,}003.5$ ns $= 6.5000035$ ms ≈ 6.5 ms.

- **LRU:** สำหรับ reference string นี้ จำนวน faults และ hits เท่ากับ FIFO จึงได้ $p = 0.65$, $h = 0.35$ และ $\text{EAT} = 6{,}500{,}003.5$ ns $= 6.5000035$ ms ≈ 6.5 ms.

- **OPT:** $p = 10/20 = 0.5$ และ $h = 10/20 = 0.5$.
  $\text{EAT} = 0.5(10\,\text{ns}) + 0.5(10{,}000{,}000\,\text{ns})$
  $= 5 + 5{,}000{,}000 = 5{,}000{,}005$ ns $= 5.000005$ ms ≈ 5.0 ms.

สรุปผล

Policy	Faults	Hits	p	EAT
FIFO	13	7	0.65	6.5000035 ms
LRU	13	7	0.65	6.5000035 ms
OPT	10	10	0.50	5.0000050 ms

OPT ให้ EAT ต่ำที่สุดในชุดนี้ เพราะเกิด page fault น้อยที่สุด. FIFO และ LRU ได้จำนวน fault เท่ากันโดยบังเอิญจาก reference string นี้ ไม่ได้หมายความว่าจะเท่ากันทุกชุด. เมื่อตอบโดยปัดเศษ ให้รายงานประมาณ 6.5 ms สำหรับ FIFO/LRU และ 5.0 ms สำหรับ OPT.

เกณฑ์ตรวจรวม 8 คะแนน

- ตาราง frame รวม 6 คะแนน: policy ละ 2 คะแนน ให้คะแนนตามสถานะ frame ทั้ง 20 คอลัมน์ คอลัมน์ละ 0.1. ไม่หักคะแนนหากไม่ได้เพิ่มแถว H/F หรือสรุปจำนวน hit/fault แยกต่างหาก.
- EAT รวม 2 คะแนน: ตั้งสูตร 0.5 คะแนน, แปลงหน่วย 0.5 คะแนน และคำนวณ EAT ของแต่ละ policy ข้อละ $1/3$ คะแนน. ไม่ต้องเขียนข้อสรุปเพิ่มหากโจทย์ไม่ได้ถาม.

หากตารางก่อนหน้ามีข้อผิดพลาด แต่ผู้สอบนำจำนวน fault ของตนไปคำนวณ EAT ต่อได้ถูกวิธี ให้คะแนนขั้นคำนวณต่อเนื่อง โดยหักข้อผิดพลาดเดิมที่ตารางหรือจำนวน fault เท่านั้น. ห้ามให้คะแนนเต็มหากนำค่าคนละหน่วยมาบวกกันในสมการเดียว.

F. จุดที่ควรยืนยันกับอาจารย์ และการตรวจความครบถ้วน

ข้อที่ต้นฉบับหรือเครื่องหมายเฉลยยังมีความกำกวม

- **ปรนัย 1 (แหล่ง E09 Q1):** โจทย์เรียก trap ว่า system call. ในเนื้อหาวิชา system call เป็นคำขอบริการ ส่วน trap เป็นกลไกที่อาจใช้ส่งการควบคุมเข้า OS; เฉลยคงตัวเลือกที่ทำเครื่องหมายไว้ แต่ควรถามว่าต้องการใช้สองคำนี้แทนกันหรือไม่.
- **ปรนัย 5 (E09 Q6):** stem ระบุ non-preemptive แต่ตัวเลือกที่ทำเครื่องหมายคือ timer interrupt ซึ่งใช้กับการแย่ง CPU ในระบบ preemptive. ควรถามว่าคำว่า non-preemptive พิมพ์คลาดเคลื่อนหรือไม่.
- **ปรนัย 16 (E09 Q15):** ``ทำงานตามลำดับที่มาถึง" อาจหมายถึงเลือกงานถัดไปแบบ FIFO หรือหมายถึงลำดับที่งานได้ CPU ในแต่ละรอบ ซึ่งทำให้ RR ถูกตีความรวมได้. เฉลยคง FIFO ตามเครื่องหมายเดิมและระบุการตีความไว้.
- **ปรนัย 17 (E09 Q19):** ข้อสอบเก่าทำเครื่องหมาย register ว่าเร็วที่สุด แต่ C ไม่รับประกันว่า compiler จะเก็บตัวแปรไว้ใน register หรือทำให้เร็วกว่าเสมอ. เฉลยเก็บคำตอบตามแหล่งและบอกข้อจำกัด.
- **ปรนัย 22 (E09 Q24):** เฉลยต้นฉบับทำเครื่องหมาย (d) แต่ external fragmentation ใน (c) ก็เป็นข้อเสียของ contiguous allocation ได้. จึงไม่ควรยืนยันว่ามีคำตอบเดียวโดยไม่มีคำชี้แจงเพิ่ม.
- **ปรนัย 41 (E09 Q50):** ``not valid in page table" แยกไม่ชัดระหว่าง virtual page ที่ถูกต้องแต่ยังไม่ present กับ address ที่ผิดกฎหมาย. เฉลยตีความตามเครื่องหมายว่าเป็นกรณี page ยังไม่อยู่ใน RAM.
- **ปรนัย 13 (E09 Q14):** ผลว่า parent พิมพ์หลัง child ขึ้นกับโปรแกรม lab ที่อ้างถึง; การเรียก wait() อย่างเดียวบอกได้เพียงว่า parent รอจน child จบ.
- **อัตนัยข้อ 1:** โจทย์ไม่ได้กำหนด tie-break เมื่องาน SJF ยาวเท่ากัน. เฉลยใช้ลำดับมาถึง และควรรับคำตอบที่ใช้ tie-break อื่นอย่างถูกต้องด้วย.
- **อัตนัยข้อ 3:** ใช้สูตร EAT ตาม Lecture 6 และต้นทุนที่โจทย์ให้เท่านั้น; ไม่เพิ่มเวลาอื่นของ page-fault handling ที่โจทย์ไม่ได้ระบุ.

ตรวจความครบถ้วนเทียบกับข้อสอบจำลอง

เฉลยมีคำตอบครบทั้งปรนัย 41/41 ข้อ, ตอบสั้น 10/10 ข้อ และอัตนัย 3/3 ข้อ. ข้อเขียนข้อ 1 มี Gantt chart ของ FIFO/SJF/RR, turnaround และ response time ราย process, ค่าเฉลี่ย และเกณฑ์ตรวจ; ข้อ 2 มีตัวอย่างคำตอบ, กฎ MLFQ ทั้งห้าข้อ, ตัวอย่างเดินคิว, ข้อเสนอปรับปรุงและข้อจำกัด; ข้อ 3 มีตาราง frame ครบ 20 references สำหรับ FIFO/LRU/OPT, จำนวน hit/fault, เหตุผลเลือก victim และ EAT พร้อมแปลงหน่วย. คะแนนย่อยรวม $8 + 4 + 8 = 20$ และทั้งฉบับรวม 50 คะแนน.

การตรวจนี้ยืนยันความครบถ้วนเมื่อเทียบกับข้อสอบจำลองชุดนี้ ไม่ได้หมายความว่าครอบคลุมทุกหัวข้อในสไลด์ Lecture 1--6. ข้อความในภาพประกอบที่ไม่เกี่ยวกับโจทย์และไม่ชัดจากการดึงข้อความไม่ได้ถูกเดาหรือเติมเพิ่ม; จุดกำกวมที่มีผลต่อคำตอบระบุไว้ด้านบน.

แผนที่เนื้อหาและหน้า PDF ของแหล่งหลัก

รหัส L1--L6 ที่อยู่ท้ายเฉลยหมายถึงหน้า PDF ในไฟล์ต่อไปนี้ (เลขหน้าเป็นเลขหน้า PDF):

- L1 = lectures/Lecture1_IntroductionToOS.pdf: trap และ privilege p. 2; isolation p. 11; system call และ memory protection pp. 13--15. ครอบคลุมปรนัย 1, 2, 4.
- L2 = lectures/Lecture2_ProcessAndProcessAPIs.pdf: process components p. 4; states pp. 6--8; PCB pp. 10--12; context switch pp. 22--24; cooperative/yield p. 27; timer interrupt p. 29; fork pp. 34--36; wait pp. 37--38; termination/exit status pp. 40. ครอบคลุมปรนัย 3, 5--15, 35--36 และตอบสั้น 2, 4--6.
- L3 = lectures/Lecture3_Scheduling.pdf: metrics pp. 6, 13; FIFO p. 7; SJF pp. 9--10; STCF p. 12; RR pp. 14--16; MLFQ pp. 24--35. ครอบคลุมปรนัย 16, 19, 27--28, 30--31, 37--38; ตอบสั้น 1, 7; อัตนัยข้อ 1--2.
- L4 = lectures/Lecture4_MemoryConceptAndContiguousMemoryAllocation.pdf: C storage classes pp. 34--35; base/bound pp. 17--18; fixed/flexible partitions pp. 20--21; internal fragmentation p. 27; external fragmentation and compaction pp. 29--31. ครอบคลุมปรนัย 7, 17--18, 22, 32; ตอบสั้น 9.
- L5 = lectures/Lecture5_NoncontiguousMemoryAllocation.pdf: segmentation pp. 5--10; paging/page table pp. 12--19; address translation pp. 14--16; TLB pp. 20--22. ครอบคลุมปรนัย 21--22, 25--26, 33--34, 39; ตอบสั้น 3, 10.
- L6 = lectures/Lecture6_VirtualMemory.pdf: virtual memory/page mapping pp. 3--9; page fault/handler pp. 10--11; page selection/demand paging pp. 12--13; FIFO/LRU/OPT pp. 13--24; EAT p. 29; working set/thrashing pp. 33--34. ครอบคลุมปรนัย 20, 23--24, 29, 40--41; ตอบสั้น 8; อัตนัยข้อ 3.

แหล่งข้อสอบที่ใช้ตรวจข้อความและเฉลย

E09 หมาย ถึง quiz-and-exam/ midterm/ os midterm exam.pdf, หน้า 1--16; ปรนัย 40 ข้อ อิง คลัง คำถาม นี้ โดย ตรวจ โจทย์ และ ตัว เลือก ที่ ต่อ ข้าม หน้า. E03 หมาย ถึง quiz-and-exam/ midterm/ 20181029161937_CPE_326_Operating_System_2561.pdf; ใช้เทียบโจทย์ fork หนึ่งข้อ. ข้อเขียนเทียบกับ quiz-and-exam/midterm/ยำรวมมิตรข้อเขียน.pdf, pp. 4--6, 10. ต้นฉบับโจทย์ที่เฉลยชุดนี้ตอบอยู่ใน quiz-and-exam/mock-exam/midterm/output/pdf/CPE333_Midterm_Mock_Exam.pdf.

Lecture PDF บางหน้าเป็นภาพสไลด์ที่มีตัวหนังสือประกอบหรือ annotation ซึ่งดึงเป็นข้อความไม่ได้ครบ. ใช้เฉพาะแนวคิดและตัวเลขที่ตรวจอ่านได้จากแหล่งหลัก; หากเฉลยจากข้อสอบเก่าขัดกับหลักใน Lecture จะระบุความต่างแทนการแก้หรือเดาเงียบ ๆ.

คำอธิบายเสริม: เฉลยคำถามฝึกทบทวน 10 ข้อ

1. Program คือชุดคำสั่งที่เก็บไว้และยังไม่ทำงาน; process คือการทำงานของ program หนึ่ง instance พร้อมข้อมูลและสถานะของมัน.

2. เปลี่ยนจาก ready ไป blocked โดยตรงไม่ได้. process ต้องได้ CPU และเข้าสู่ running ก่อน แล้วจึง block เมื่อร้องขอ I/O หรือรอเหตุการณ์; เมื่อเหตุการณ์เสร็จจึงกลับไป ready.

3. ตัวอย่างค่าที่บันทึก ได้แก่ program counter และ stack pointer. PCB เก็บข้อมูลที่ OS ต้องใช้ติดตามและจัดการ process เช่น state, PID, scheduling information และ saved CPU context.

4. FIFO ทำงาน A, B, C จึงมี turnaround 4, 6, 7 ms และค่าเฉลี่ย $(4 + 6 + 7)/3 = 17/3 \approx 5.67$ ms. SJF ทำงาน C, B, A จึงมี turnaround 1, 3, 7 ms และค่าเฉลี่ย $(1 + 3 + 7)/3 = 11/3 \approx 3.67$ ms.

5. Response time $= 5 - 2 = 3$ ms. Turnaround time $= 9 - 2 = 7$ ms.

6. MLFQ เลือกคิว priority สูงสุดก่อน และใช้ RR เมื่อมีหลายงาน priority เท่ากัน. Priority boost เป็นระยะช่วยให้งานที่อยู่คิวล่างนานกลับมามีโอกาสใช้ CPU; การคิดเวลาตาม allotment สะสมช่วยป้องกันการหลบกฎด้วยการ yield ก่อนหมด slice.

7. ก่อนอ้างถึง page 4 มี page 1, 2, 3 อยู่ใน frame. FIFO ยังถือว่า page 1 เก่าที่สุด จึงนำ page 1 ออก. LRU รู้ว่า page 1 เพิ่งถูกอ้างถึง จึงนำ page 2 ซึ่งไม่ได้ถูกใช้นานที่สุดออก.

8. 16 KiB $= 16{,}384$ bytes และ 4 KiB $= 4{,}096$ bytes. เนื่องจาก $0 \leq 3000 < 4096$, address ใช้ได้; physical address คือ $16{,}384 + 3000 = 19{,}384$ bytes.

9. TLB miss หมายถึงไม่พบ translation ใน cache; hardware/OS ยังตรวจ page table ได้. ถ้า page table ระบุว่า page อยู่ใน RAM ก็แปล address ต่อได้โดยไม่เกิด page fault. Page fault เกิดเมื่อ page ที่ต้องใช้ยังไม่อยู่ใน RAM.

10. $d_a = 10$ ms $= 10{,}000{,}000$ ns.
    $EAT = (1 - 0.001)(10) + 0.001(10{,}000{,}000) = 9.99 + 10{,}000 = 10{,}009.99$ ns $= 10.00999$ μs ≈ 10.01 μs.